AI Engineering Toolkit

MCP Servers & Companion Tools Beyond Claude Code

September 2026

AI Engineering Toolkit

MCP Servers & Companion Tools Beyond Claude Code's Core Docs

Context7 was one example. This is the wider map: what exists, what it's for, and when to actually reach for it instead of building something yourself.

Why this matters

Claude Code's own docs cover the mechanism (MCP, skills, hooks). They don't catalog the ecosystem that's grown on top of it. As of 2026 there are 17,000+ public MCP servers. Most of what you need already exists — the skill is knowing *when* to reach for one, not building it.

1. Documentation & Context Servers

These fix one specific problem: a model's training data goes stale, but library APIs keep changing. Instead of guessing, the model asks a server for the current answer.

Server	What it does	Add it
Context7 (Upstash)	Fetches current, version-specific docs for any popular library straight into context	`claude mcp add --transport http context7 https://mcp.context7.com/mcp`
DeepWiki MCP	Lets Claude ask natural-language questions against any GitHub repo's structure and code, not just search it	Connect via the DeepWiki-hosted MCP endpoint
Ref (ref.tools)	Alternative to Context7, similar doc-search-into-context approach	Hosted HTTP endpoint, add like any remote server

When to use one: any time you catch Claude confidently writing code against an API version that's since changed, or hallucinating a method that doesn't exist. Append `use context7` (or the equivalent trigger phrase) to the prompt, or put a standing rule in CLAUDE.md: "always use context7 for library-specific questions."

2. Database & RAG-Specific Servers

Directly relevant if you're building a RAG pipeline — these let Claude inspect and query the actual retrieval layer instead of you copy-pasting schema or query results into chat.

Server	What it does
Postgres / DBHub	Natural-language queries, schema exploration, read-only by design when you scope the connection string
Supabase MCP	Official server — schema, queries, auth config, storage, all from one connection
Pinecone MCP	Inspect indexes, run similarity queries, check embedding stats directly from a session
Chroma MCP	Same idea for a local/self-hosted Chroma vector store
Weaviate MCP	Same idea for Weaviate deployments
Vectara MCP	Managed semantic-search-as-a-service, exposed as retrieval tools rather than raw vector ops

Practical pattern for a RAG project: connect your vector DB's MCP server, then ask Claude directly — "show me the top 5 chunks retrieved for this query and their similarity scores" — instead of writing a one-off debug script every time you want to inspect retrieval quality.

> Use a read-only credential in any database connection string you hand to an MCP server. Claude's queries should never be able to write to production data unless that is explicitly the point.

3. Web Data & Scraping

Server	What it does
Firecrawl MCP	Search, scrape, crawl, and structure live web pages into clean data for an agent — keyless trial available, API key for full access
Playwright MCP	Full browser automation — navigate, click, fill forms, read rendered output
Puppeteer MCP	Similar browser automation, older/more established tooling

When to use one: your agent needs live data that isn't in any API — competitor pricing, a page behind JavaScript rendering, or you're testing your own web app's actual rendered behavior (relevant for your mobile-optimization and platform work).

4. Dev Workflow & Issue Tracking

Server	What it does
GitHub MCP	Repos, PRs, commits, issues — permission-aware, the same access level as your token
Linear MCP	Create/update issues, query sprint status, from inside a coding session
Jira / Confluence (Atlassian official)	46+ tools across issue tracking and documentation search
Notion MCP	Read/write workspace pages and databases as live context
Sentry MCP	Pull error and monitoring data directly into a debugging session — no more copy-pasting stack traces

When to use one: the trigger from your reference PDF still applies — “you keep copying data from a tab Claude can’t see.”

5. Observability for Agentic Pipelines

This is the category most relevant to what you're actually building — a multi-step agent pipeline is much harder to debug than a single Claude Code session, because you need to see *why* a chain of steps produced a bad output.

Tool	What it does	MCP?
LangSmith	Tracing, evaluation, and dataset management for multi-step LLM chains	No — standalone platform
Langfuse	Open-source alternative to LangSmith — tracing, prompt management, cost tracking	No — standalone, self-hostable
Arize Phoenix	Open-source tracing via OpenTelemetry; plugs directly into the Agent SDK's instrumentation hooks	No — OTel-based

None of these are MCP servers — they're the standard tooling for answering "which step in my agent pipeline actually went wrong" once you move past interactive Claude Code sessions into production agents built with the Agent SDK.

6. Orchestration Frameworks (complementary, not MCP)

If you're building agent logic that lives outside Claude Code entirely — a scheduled pipeline, a backend service — these are the frameworks people reach for alongside or instead of the raw Agent SDK:

Framework	What it's for
LangChain	General-purpose LLM app framework — chains, memory, tool-calling abstractions
LangGraph	Graph-based control flow for multi-step/multi-agent systems, built by the LangChain team
LlamaIndex	RAG-specific — indexing, retrieval, and query engines, often paired with an MCP layer on top

Where the Agent SDK fits relative to these: the SDK gives you Claude Code's own agent loop, tools, and context management as a library. LangChain/LangGraph/Ll are model-agnostic frameworks you'd reach for if you want more explicit control over multi-agent graph structure, or need to swap models later. For most single-agent production pipelines built around Claude specifically, the Agent SDK alone (chapter 18 of your Claude Code reference) is the simpler starting point.

7. Decision Table — What Do I Actually Need?

You’re building…	Reach for
A coding assistant that needs current library docs	Context7 or Ref
A RAG chatbot’s retrieval debugging	Your vector DB’s MCP server (Pinecone/Chroma/Weaviate)
An agent that needs live web data	Firecrawl or Playwright MCP
A dev-workflow copilot (PRs, issues, docs)	GitHub + Linear/Jira + Notion MCP
A production agent pipeline you need to debug	Langfuse or Arize Phoenix, wired into the Agent SDK
A multi-agent system with complex branching logic outside Claude Code	LangGraph or a dynamic workflow (see chapter 10)
Something that already has 17,000 existing servers doing it	Search before you build — check the Anthropic Directory or a marketplace first

8. Quick Setup Reference

```
# Documentation
claude mcp add --transport http context7 https://mcp.context7.com/mcp

# Database (read-only credential!)
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub \
  --dsn "postgresql://readonly:pass@host:5432/dbname"

# GitHub
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"

# Web scraping
claude mcp add --transport http firecrawl https://mcp.firecrawl.dev/mcp \
  --header "Authorization: Bearer YOUR_FIRECRAWL_KEY"
```

Verify any connection with `/mcp` before relying on it — check that it shows **connected**, not just that the add command printed success.

One caution worth repeating

Every server on this list can read data into Claude’s context, and some can act on your behalf (create issues, run queries, push commits). Verify you trust a server before connecting it — a server that fetches external content, including documentation servers, is a potential prompt-injection vector. Scope credentials to read-only wherever the task allows it, especially for anything touching production data.